Clinic Medical Assistant RAG — Professional Build Plan

Maps every concept from DeepLearning.AI's RAG course (Module 5: Production) onto your clinic bot project.

---

1. Folder Structure

```
clinic_rag_bot/
├── config/
│   └── settings.yaml          # model names, thresholds, paths — never hardcode these
├── data/
│   ├── patients.json
│   ├── medications.json
│   └── test_questions.json
├── src/
│   ├── __init__.py
│   ├── config_loader.py       # loads settings.yaml into a Config object
│   ├── ingestion.py           # PatientDataLoader, MedicationDataLoader, DocumentChunker
│   ├── embeddings.py          # EmbeddingModel (text + multimodal wrapper)
│   ├── vector_store.py        # VectorStore class (wraps Chroma)
│   ├── retrievers.py          # LocalRAGRetriever, PubMedRetriever
│   ├── router.py              # QueryRouter
│   ├── generator.py           # AnswerGenerator (LLM call wrapper)
│   ├── safety_agent.py        # SafetyAgent
│   ├── pipeline.py            # ClinicRAGPipeline — orchestrates everything
│   ├── logger.py              # structured logging setup
│   └── tracer.py              # request tracing (per-step timing/logging)
├── evaluation/
│   ├── evaluator.py           # Evaluator class — runs test_questions.json, scores results
│   └── results/               # saved evaluation runs (timestamped)
├── bot/
│   └── telegram_bot.py        # thin adapter — imports pipeline, nothing else
├── logs/
│   └── app.log
├── .env                       # API keys — NEVER commit this
```

```
├── requirements.txt
└── main.py
```

This separation is the "professional" part mentors notice immediately: **the bot file has zero business logic** — it only calls `pipeline.process(message)`.

---

2. Class Architecture

`Config` (config_loader.py)

Loads `settings.yaml`. Holds: model names, similarity thresholds, top-k values, log level, escalation keyword list. Nothing in the codebase hardcodes a model name or magic number — everything reads from here.

yaml

```yaml
# config/settings.yaml
models:
  generation: "gemini-2.5-flash-lite"
  safety_agent: "gemini-2.5-flash"   # more careful model for the safety check
  text_embedding: "text-embedding-3-small"
  image_embedding: "clip-ViT-B-32"

retrieval:
  top_k: 4
  similarity_threshold: 0.55   # below this → treat as "no confident match"

safety:
  hard_escalation_keywords: ["chest pain", "can't breathe", "bleeding heavily",

logging:
  level: "INFO"
  log_file: "logs/app.log"
```

`EmbeddingModel` (embeddings.py)

Wraps both a text embedder and (for the multimodal stretch goal) a CLIP-style image embedder. Single `.embed(item, modality="text"|"image")` interface — this is directly the course's "multimodal embedding model" concept: same vector space, different modalities.

`VectorStore` (vector_store.py)

Wraps Chroma. Methods: `add_documents()`, `query(vector, top_k)`, `add_images()` (stretch

goal). Keeps two collections: `clinic_data` and `pubmed_cache`.

`LocalRAGRetriever` / `PubMedRetriever` (retrievers.py)

Each implements a shared interface: `.retrieve(query) -> List[RetrievedChunk]`. `RetrievedChunk` is a small dataclass: `{text, source, score}`. This uniformity means your `QueryRouter` doesn't care which retriever it's calling.

`QueryRouter` (router.py)

Classifies incoming query into `clinic_specific | general_medical | emergency`. Implementation: one LLM call with a strict JSON-output prompt. Logs its own decision + confidence.

`AnswerGenerator` (generator.py)

Takes `(query, retrieved_chunks)` → calls the LLM → returns draft answer. This is where **quantization/cost/latency tradeoffs** (course concept) become a real decision: you pick Flash-Lite here for cost, but document why.

`SafetyAgent` (safety_agent.py)

Two-layer design (this is your strongest talking point for mentor review):

1. **Hard rule layer** — keyword/pattern match against `config.safety.hard_escalation_keywords`. Deterministic, no LLM, cannot be "argued out of" by a clever prompt.
2. **LLM judgment layer** — a second, more careful model (per config) scores risk 0-1 on anything that passes layer 1. Also checks retrieval confidence — low confidence → escalate rather than guess.

Returns a dataclass: `SafetyDecision(action, risk_level, reason)`.

`ClinicRAGPipeline` (pipeline.py)

The orchestrator. `process(message) -> str`. Internally: Router → Retriever(s) → Generator → SafetyAgent → final formatted reply. Every step is wrapped by the `Tracer`.

`Tracer` (tracer.py)

Records per-request: timestamp, which route was taken, retrieval latency, generation latency, safety decision, total latency. This is your **logging/monitoring/observability** course module, implemented directly — write each trace as a JSON line to `logs/traces.jsonl`.

`Evaluator` **(evaluation/evaluator.py)**

Loads `test_questions.json`, runs each through the pipeline, compares `expected_action`/`expected_route` against actual, and outputs a scored report (accuracy per category, and critically: **zero tolerance flag** if any emergency question wasn't escalated). This is your **evaluation strategies** course module.

---

3. Task List (in build order)

Phase 1 — Foundations

- [ ] Set up folder structure + `requirements.txt` + `.env` (API keys, gitignored)
- [ ] Build `Config` + `config_loader.py`, load `settings.yaml`
- [ ] Build `logger.py` — structured logging to console + file, with levels

Phase 2 — Data + Retrieval

- [ ] Build `PatientDataLoader` / `MedicationDataLoader` (ingestion.py) — load JSON, chunk into text docs
- [ ] Build `EmbeddingModel` (text-only first, image support stretch goal)
- [ ] Build `VectorStore` wrapping Chroma, two collections
- [ ] Build `LocalRAGRetriever` — test retrieval standalone before wiring anything else
- [ ] Build `PubMedRetriever` (Entrez API + local cache)

Phase 3 — Reasoning Layer

- [ ] Build `QueryRouter` — test against 10 sample questions, log its classification + confidence
- [ ] Build `AnswerGenerator` — wire retriever output into prompt, test in isolation
- [ ] Build `SafetyAgent` — hard-rule layer first (test against all `test_questions.json` emergency cases), then add LLM judgment layer

Phase 4 — Orchestration + Observability

- [ ] Build `ClinicRAGPipeline` — wire Router → Retriever → Generator → SafetyAgent
- [ ] Build `Tracer` — wrap pipeline steps, write `logs/traces.jsonl`
- [ ] Manually inspect 5-10 traces — do the timings/decisions make sense?

Phase 5 — Evaluation

- [ ] Build `Evaluator` — run full `test_questions.json` set through pipeline

- [ ] Generate scored report: per-category accuracy, safety-agent miss rate (must be 0 on emergency category)
- [ ] Save timestamped run to `evaluation/results/`

Phase 6 — Interface + Deployment

- [ ] Build `telegram_bot.py` — thin adapter calling `pipeline.process()`
- [ ] Add typing indicator, error handling (pipeline exceptions → safe fallback message, never crash the bot)
- [ ] Manual end-to-end test on Telegram

Phase 7 — Stretch Goals (pick what you have time for)

- [ ] Multimodal: accept a photo of a medication box or lab report, embed with CLIP, retrieve against it
- [ ] Cost/latency logging: track token usage per request, log to trace file, summarize $ cost per day
- [ ] Security pass (course's Security lesson): sanitize user input before it hits any prompt (prompt injection defense), never let retrieved patient data leak to the wrong requester, rate-limit per user

---

4. What Makes This "Professional" for Mentor Review

1. **Separation of concerns** — bot file has no logic, pipeline has no Telegram code
2. **Config-driven, not hardcoded** — swapping models or thresholds is a one-line YAML edit
3. **Observability** — you can show a trace log proving what happened on any given request, not just "trust me it works"
4. **Evaluation, not vibes** — a scored report against known test cases, with the safety-critical category held to zero tolerance
5. **Documented tradeoffs** — you can explain *why* Flash-Lite for generation but full Flash for the Safety Agent (cost vs. reliability), and *why* two-layer safety (deterministic + LLM judgment) instead of LLM-only

This structure directly mirrors what Module 5 of your course teaches — you're not just using the ideas, you're implementing each one as a distinct, inspectable piece of the system.